Introduction

If you've ever built an n8n workflow that quietly did *less* than you expected — sent one Slack message instead of fifty, updated one row instead of a hundred — you've run into one of the most common and least understood behaviors in node-based automation: **n8n executes per item, not per node.**

This isn't a bug. It's the execution model. But if you don't understand it, it looks exactly like a bug, and it's the kind that doesn't throw an error. It just silently does the wrong amount of work. That makes it dangerous in production — the workflow "succeeds," the logs look clean, and you only find out 49 orders never got processed when someone asks why.

This matters beyond n8n, too. The mental model here — "does this step run once, or once per element in a collection" — is the same question you ask when writing a `for` loop, mapping over an array in JavaScript, or reasoning about a batch job in any backend system. Understanding it in n8n makes you a better engineer everywhere, and understanding it in code makes n8n click immediately. That overlap is exactly why this kind of problem shows up in interviews and in production incidents alike: the underlying skill is recognizing when you have "one thing" versus "many things," and knowing which operation you actually need.

Problem Overview

Here's the setup. You have an HTTP Request node that calls an API and gets back a response. That response is a single item — but nested inside it is an array field, say `orders`, containing 50 individual orders.

Now you drag a Slack node after it, expecting it to fire once per order. Instead, it fires exactly once — for the first order — and stops. The other 49 are never touched. No error. No warning. The workflow reports success.

The core issue is a mismatch between two different shapes of data:

- **One item containing an array** — a single envelope with 50 letters stuffed inside it.
- **Many items, one per array element** — 50 separate envelopes, each with one letter.

Every node downstream operates on items, not on whatever's nested inside them. If your array is buried inside one item, nodes downstream see *one thing to process*, not fifty.

Example

Imagine the HTTP Request node returns this:

```
[
  {
    "customer": "Acme Corp",
    "orders": [
      { "id": 1, "total": 80 },
      { "id": 2, "total": 95 },
      { "id": 3, "total": 120 }
      // ...47 more
    ]
  }
]
```

That's **one item**. The `orders` array lives *inside* it. A Slack node placed right after this will run once, using whatever it can find on that single item — and if it's mapped to reference the first order, that's all it will ever send.

Now compare that to what you actually want: 50 separate items, each shaped like `{ "id": 1, "total": 80 }`. That's the shape a downstream node needs in order to run once per order.

Intuition

The way to think about this isn't "n8n is confusing," it's "a node is a mail-sorting machine that handles one envelope at a time." Give it one envelope with 50 letters crammed inside, and it sorts the envelope — once. It never opens it up and looks at the 50 letters individually, because as far as the machine is concerned, there's one envelope in front of it.

Once you see it that way, the fix designs itself. You don't need custom logic to loop over the array — you need a machine that takes one envelope and outputs 50 separate envelopes, one per letter. And later, when you're done processing each one, you need a machine that does the reverse: collects 50 envelopes back into one.

n8n ships both of these as built-in nodes, and the trick is realizing they're a matched pair — Split Out breaks one item into many, Aggregate collects many items into one. Nothing in between needs a Function node or a line of JavaScript.

Brute Force Solution

Before reaching for the built-in nodes, it's worth understanding what the "manual" approach would look like, because it explains why n8n gives you these primitives in the first place.

Idea: Drop a Function/Code node right after the HTTP Request, write a JavaScript loop that iterates the `orders` array, and manually construct a new list of items — one per order — using n8n's item-array format (`return orders.map(o => ({ json: o }))`).

Algorithm: 1. Read the input item's `orders` field. 2. Loop over it. 3. Wrap each order as its own item object. 4. Return the new array of items.

Advantages: - Full control — you can transform, filter, or rename fields at the same time. - No conceptual leap required if you already think in code.

Disadvantages: - Requires JavaScript knowledge that half your team (or future you, six months later) may not have. - Every workflow reinvents the same 4 lines of splitting logic. - Doing the reverse (collapsing many items back to one) is a second custom loop with its own edge cases (empty array, key collisions). - No visual indication in the workflow of *why* the item count changes — harder to debug at a glance.

Complexity: O(n) time to split, O(n) space to hold n items instead of 1 — same as the built-in approach. The cost isn't performance, it's maintainability and readability.

Optimal Solution

The optimal approach uses the two nodes n8n built specifically for this: **Split Out** and **Aggregate**.

Split Out takes one item with an array field and turns it into many items — one per array element.

```
Input:  1 item  →  { orders: [o1, o2, ..., o50] }
Output: 50 items → { id: 1, total: 80 }, { id: 2, total: 95 }, ...
```

You configure it by pointing at the field name (`orders`). That's the entire setup — no code.

Aggregate does the exact opposite: many items in, one item out, with the specified fields collected into a list.

```
Input:  50 items → { total: 80 }, { total: 95 }, ...
Output: 1 item   → { total: [80, 95, ..., 4200-worth of totals] }
```

Here's the pattern that ties it together, and the one worth memorizing because you'll reuse it constantly:

Stage	What happens	Item count
HTTP Request	Fetch data, array buried in one item	1
Split Out	Explode array into individual items	50
Process (Slack, DB write, charge, etc.)	Runs once per item	50
Aggregate	Collapse results back into one summary item	1

Two details matter enough to call out separately, because they're the parts that bite people silently:

- **Empty arrays kill the chain.** If `orders` comes in empty, Split Out produces zero items — and zero items means everything after it, including Aggregate, simply never executes. No error, no summary, nothing runs. If you need a "processed 0 orders" message, you have to check for the empty case *before* Split Out and branch explicitly.
- **Aggregate does field-name matching, not logic.** It doesn't reconcile which total belongs to which order — it just grabs the exact field name you give it from every item and stacks the values into a list. A typo, a renamed field, or a field that only exists on some items won't throw an error. You'll just get a quietly empty or partial array in the output.

Python Code

While Split Out and Aggregate are no-code nodes, the underlying operation is worth seeing in Python, because it's the exact same shape of problem you'll hit whenever you're transforming data between "one record with a nested list" and "many records, one per list element."

```
from typing import Any


def split_out(item: dict[str, Any], field: str) -> list[dict[str, Any]]:
    """Equivalent of n8n's Split Out: one item with an array field
    becomes many items, one per array element."""
    values = item.get(field, [])
    return [value for value in values]


def aggregate(items: list[dict[str, Any]], field: str) -> dict[str, Any]:
    """Equivalent of n8n's Aggregate: many items become one item
    holding the collected field values as a list."""
    collected = [entry[field] for entry in items if field in entry]
    return {field: collected}


def process_orders(source_item: dict[str, Any]) -> dict[str, Any]:
    orders = split_out(source_item, "orders")
```

```
    if not orders:
        return {"total": [], "count": 0}

    # simulate the "process each order" step (charge, log, etc.)
    processed = orders

    summary = aggregate(processed, "total")
    summary["count"] = len(processed)
    return summary
```

Code Walkthrough

- `split_out` mirrors the Split Out node: it reads the named field off a single item and returns its contents as a standalone list — the "one envelope becomes many" step.
- `aggregate` mirrors the Aggregate node: it walks a list of items and pulls out just the named field from each, collecting them into a single list under that same key. Note the `if field in entry` guard — this is exactly the silent-failure behavior from the real node: a missing field just gets skipped, not flagged.
- `process_orders` is the full pipeline: split, process, aggregate — the same three-stage pattern from the workflow. The `if not orders` check is the piece n8n *doesn't* give you for free; it's the explicit empty-array guard you have to build yourself if you want a "zero processed" result instead of silence.

Dry Run

Input:

```
source_item = {
    "customer": "Acme Corp",
    "orders": [
        {"id": 1, "total": 80},
        {"id": 2, "total": 95},
        {"id": 3, "total": 120},
    ],
}
```

1. `split_out(source_item, "orders")` → returns the 3-element list of order dicts. Item count goes from 1 to 3.
2. `process_orders` treats `orders` as non-empty, so it proceeds — `processed = orders` (in the real workflow, this is where Slack/DB/charge logic would run per item).

3. `aggregate(processed, "total")` → pulls `80`, `95`, `120` from each dict → returns `{"total": [80, 95, 120]}`.
4. `summary["count"] = 3` is added.
5. Final result: `{"total": [80, 95, 120], "count": 3}` — one item, item count back down to 1.

That's the full round trip: 1 → 3 → 1, matching exactly what Split Out → process → Aggregate does inside n8n.

Complexity Analysis

- **Time:** O(n), where n is the number of elements in the array. Split Out visits each element once to emit it; Aggregate visits each item once to pull its field. There's no nested loop or comparison between items — each is correct because both operations are single linear passes with no matching logic between elements.
- **Space:** O(n) — you materialize n items where you previously had 1, and Aggregate materializes an n-length list from those n items. This is unavoidable: you can't represent "50 distinct results" in less than 50 slots of storage if you need them individually addressable at some point in the pipeline.

Alternative Solutions

Loop Over Items node (n8n's built-in batching node): Instead of Split Out, you could use n8n's "Loop Over Items" (SplitInBatches) node, which processes items in configurable batch sizes and loops back. This is the right call when you need rate-limiting or batching (e.g., "call this API 5 orders at a time" to avoid hitting a rate limit) rather than exploding everything at once. For a straightforward "one per item" fan-out with no batching concern, Split Out is simpler and has less configuration surface.

Code node with a manual loop: As covered in the brute-force section — viable if you're already comfortable in JavaScript and need to split *and* transform in the same step, but it reintroduces a maintenance and readability cost that Split Out avoids entirely for the common case.

Edge Cases

- **Empty array:** Split Out produces zero items, and everything downstream — including Aggregate — never executes. No error is thrown. You must branch and handle this explicitly before Split Out if a "zero results" message matters.

- **Field doesn't exist on the item:** Split Out with a missing field name behaves like splitting an empty array — zero output items, same silent stop.
- **Field name typo in Aggregate:** Produces an empty or partial array in the output with no error — the classic silent failure mode described above.
- **Array with a single element:** Split Out still produces exactly one item; downstream nodes run once, which is correct, but easy to mistake for "nothing changed."
- **Mixed items where the field only exists on some:** Aggregate silently drops the items missing the field rather than including a null or throwing — the resulting list will be shorter than the item count, which can be mistaken for a bug elsewhere.

Common Mistakes

1. **Assuming a node runs once per node, not once per item** — the root misunderstanding behind this entire bug class.
2. **Forgetting to guard against empty arrays**, then being confused when a workflow "does nothing" with no error in the execution log.
3. **Aggregating on the wrong field name** (a stale field, a typo, a field only present on some items) and not noticing the output array is empty or short.
4. **Reaching for a Function node by default** instead of checking whether Split Out / Aggregate already solve the problem with zero code.
5. **Not verifying item count at each step** while debugging — the item count shown in each node's output panel is the single fastest way to see where a workflow diverged from expectations.
6. **Expecting Aggregate to do matching or reconciliation** between fields across items, when it only stacks a single named field verbatim.
7. **Placing Aggregate before the processing step** instead of after — collapsing to one item before you've done per-order work defeats the purpose of splitting in the first place.

Interview Questions

- "Walk me through what happens if this API call returns an empty array — trace it through the whole pipeline."
- "How would you rate-limit this if the downstream API only allows 5 requests per second?"
- "What's the difference between 'many items become one' and 'grouping items by a key'? Does Aggregate do the second one?"
- "How would you detect, in production, that Aggregate silently dropped items due to a field mismatch?"

- "If you had to implement Split Out and Aggregate from scratch in Python, what would the function signatures look like?"

Similar Problems

- **LeetCode 118 (Pascal's Triangle)** and other list-generation problems — same instinct of "build a new list of records from a single seed structure," which mirrors Split Out.
- **LeetCode 49 (Group Anagrams)** — collapsing many items into grouped results by a key is the "smart" version of Aggregate that goes beyond simple field-stacking.
- **Flatten Nested List Iterator (LeetCode 341)** — directly analogous to Split Out: turning one nested structure into a flat sequence of individually-processable elements.
- **Any MapReduce-style problem** — Split Out is the "map" fan-out, Aggregate is the "reduce" collapse; recognizing that structure transfers directly to distributed data processing.

Key Takeaways

- n8n nodes run once per **item**, not once per **node** — internalize this and half the "silent" bugs in your workflows stop being mysterious.
- Split Out and Aggregate are exact opposites: one explodes an array into items, the other collapses items into an array.
- Empty arrays are a silent dead end — the chain after Split Out simply never runs, with no error.
- Aggregate matches by field name only, with zero reconciliation logic — a typo produces silence, not a crash.
- The pattern — split, process, aggregate — applies to any situation where an API, webhook, or CSV hands you a batch buried inside a single record.

Watch the Video

If you want to see this bug reproduced live — the Slack node firing once instead of fifty times, then fixed step by step with Split Out and Aggregate — watch the full walkthrough here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series that takes the kind of mess every developer eventually runs into — a silent bug, a confusing default, a "why is it doing that" moment — and breaks it down until it actually clicks. The goal isn't just to fix the one bug in front of you, it's to build the mental model that stops the next five bugs like it from happening.

Call To Action

If this saved you from an afternoon of debugging a silent Slack node, do three things: like the video on YouTube so more developers hitting this exact wall can find it, subscribe for the rest of the series, and subscribe to the Substack for the written breakdowns like this one. Drop a comment with what's still fuzzy about Split Out or Aggregate, or what n8n behavior you want covered next — the next lesson is usually pulled straight from those comments.

Quick Review

n8n: what's the tell that you need Split Out + Aggregate?

An item holds an array field you must process per-element then recombine — split fans out, aggregate folds back.

n8n execution model: runs per what?

Every node runs once per incoming item, not once per workflow — 50 items means 50 downstream executions.

Split Out on a 50-element array: what happens downstream?

Branch executes 50 times, one per array element — e.g. 50 separate API pings if a request node follows.

Common bug: array field left unsplit before an HTTP node

Node receives one item with a nested array instead of many items, so it only fires once instead of per-element.

Split Out vs Aggregate: relationship?

Inverse operations — Split Out expands one item's array into many items; Aggregate collapses many items back into one array.

Interview: why aggregate before sending a single summary email?

Downstream node needs one item with the full list, not N separate items each triggering N sends.

n8n Lesson 20: Item Lists / Aggregate node (splitting arrays into items and aggregating items back into one) — free Python cheat sheet